



UNIVERSITY OF AMSTERDAM
Amsterdam Business School



Machine Learning with Python

AAA Global Connect 2026: DATA Workshop

Dr. Jeroen van Raak

Outline

- Introduction to machine learning
 - What is machine learning?
 - Types of machine learning
- Supervised learning: learning from labeled examples
 - k-Nearest Neighbors
 - Decision Trees
 - Artificial Neural Networks
 - Case study: credit card fraud detection
- Unsupervised learning: discovering patterns in data
 - k-Means clustering
 - Case study: discovering NYC taxi zones

Resources

All slides, source code, and datasets used in this presentation are publicly available:

<https://github.com/vanraak/data2026>



edu.nl/y99yd

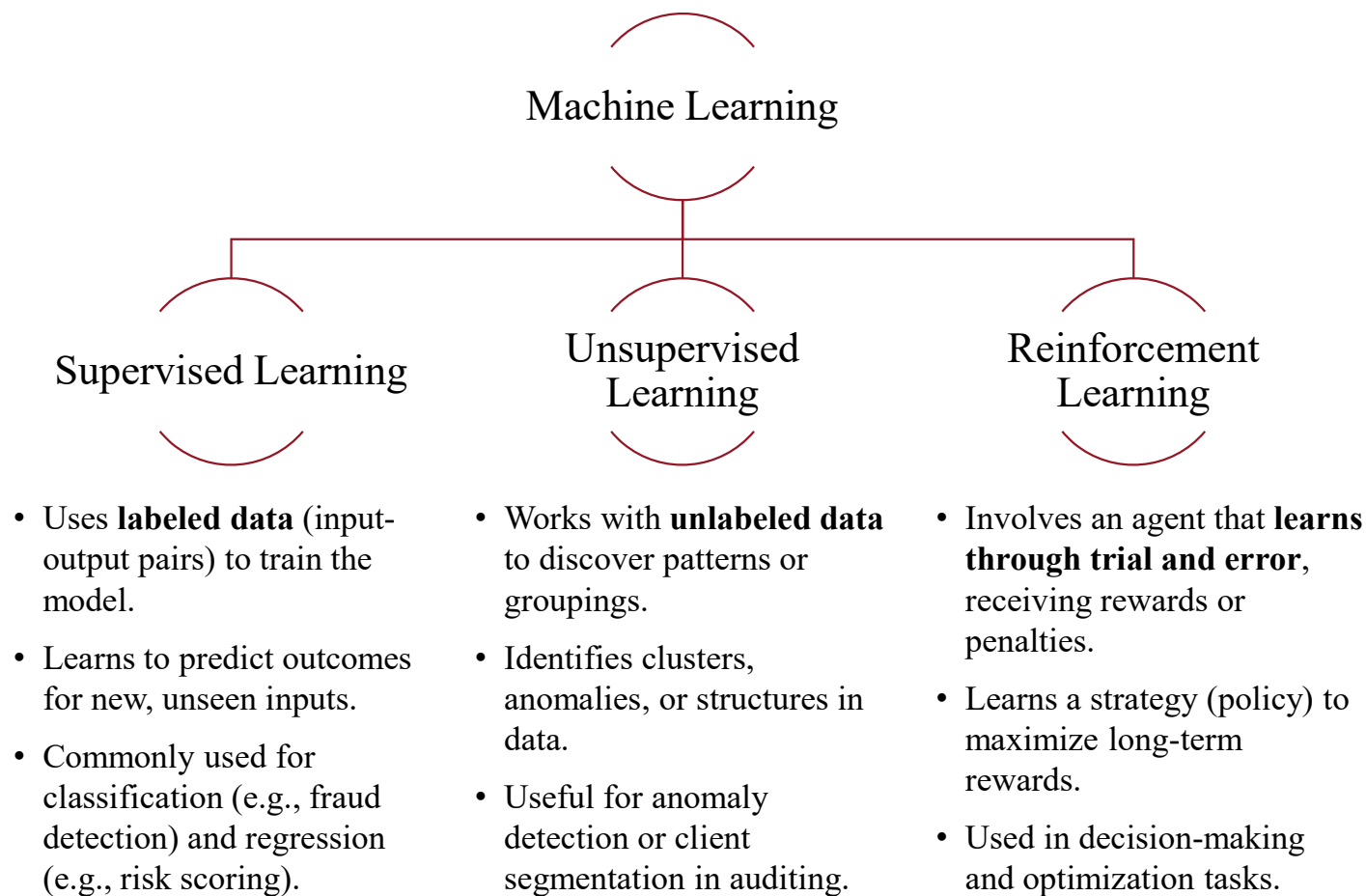
Machine Learning Applications

- Self-driving vehicles
- Automated trading
- Image recognition
- Spam filters
- Document summarization
- Chatbot creation
- Sales forecasting
- Fraud and anomaly detection
- Customer segmentation
- Product recommendations
- Forum content moderation
- Generating code, text or images

Machine Learning Definitions

- **Machine learning:** field of study that gives computers the ability to learn without being explicitly programmed (Arthur Samuel, 1959)
- **Machine learning** is the technique that improves system performance by learning from experience via computational methods. In computer systems, experience exists in the form of data, and the main task of machine learning is to develop learning algorithms that build models from data. (Zhou, 2021)

Machine Learning Paradigms



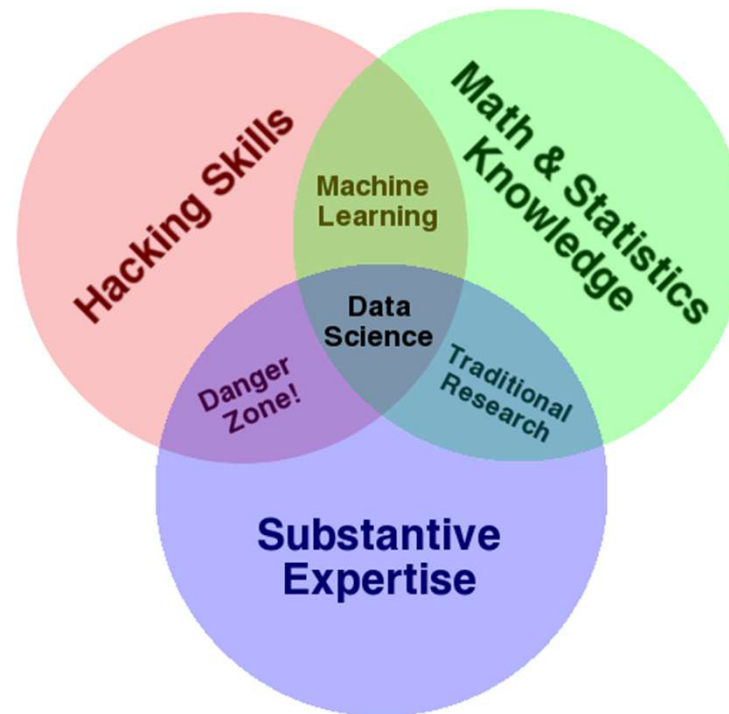
Machine Learning Algorithms

- Supervised:
 - *k*-Nearest Neighbors
 - Linear regression
 - Logistic regression
 - Support Vector Machines
 - Naive Bayes
 - Decision Trees and Random Forests
 - Neural networks
- Unsupervised:
 - Clustering (e.g., *k*-means or DBSCAN)
 - Reinforcement learning
 - Principal Component Analysis
(i.e., reduce the number of variables)

Machine Learning Strategies

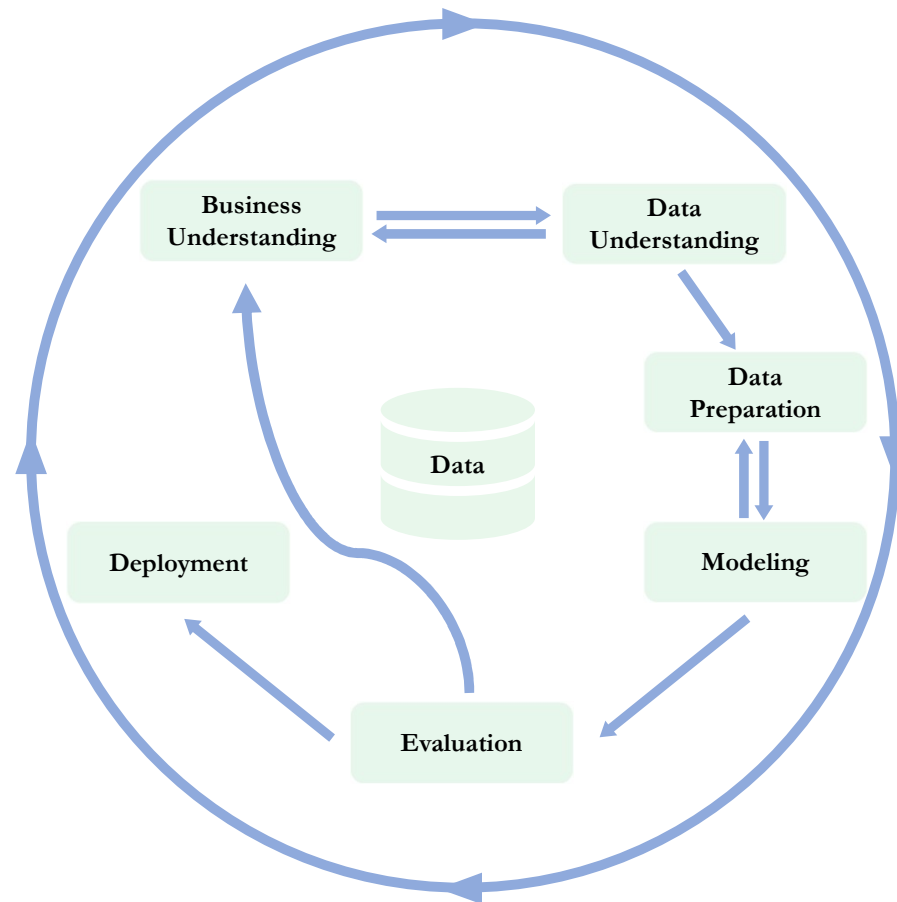
- Online vs. Batch Learning
 - Batch Learning: trains on entire dataset, heavy computation, retrain for new data
 - Online Learning: incremental updates, adapts quickly
- Instance-Based vs. Model-Based Learning
 - Instance-Based: memorizes examples, compares new data
 - Model-Based: learns general model, fast predictions, scalable

Data Science Venn Diagram



Source: <http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram>

Machine Learning Framework: CRISP-DM



Adapted from Chapman et al. (2000)

Data Preparation vs Analysis Time

■ Important distinction between tasks:

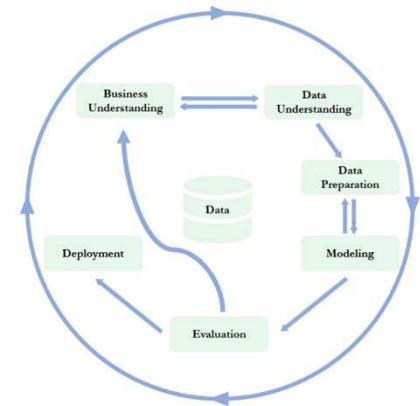
1. Data preparation: cleaning, EDA, feature engineering
2. Analysis: algorithm selection, model training, evaluation

■ Distribution of time allocated to these tasks for a typical project:



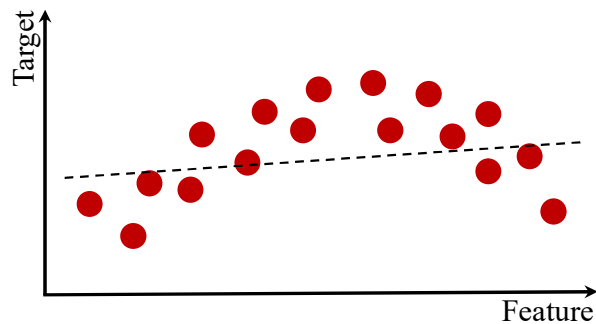
■ Preparing data is often the most time-consuming and iterative phase

- Allocate sufficient time, it directly impacts model quality



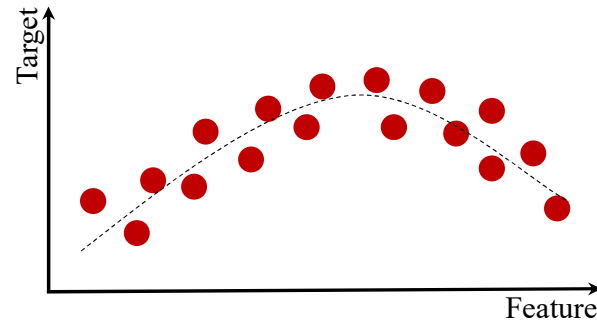
Model Complexity: Bias vs Variance

Underfitting



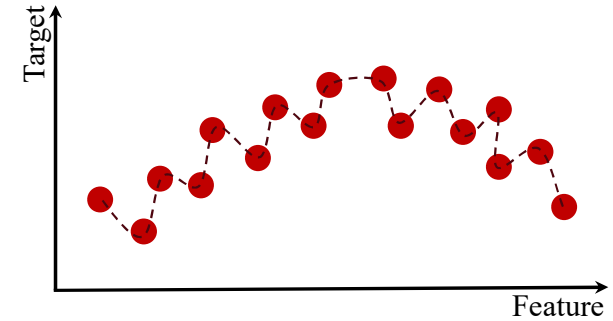
High Bias

Good fit



Balanced

Overfitting

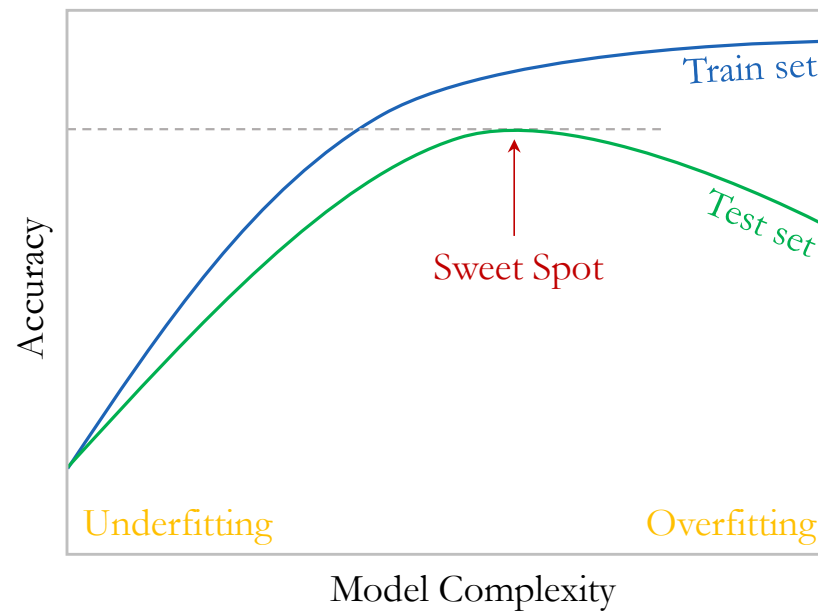


High Variance

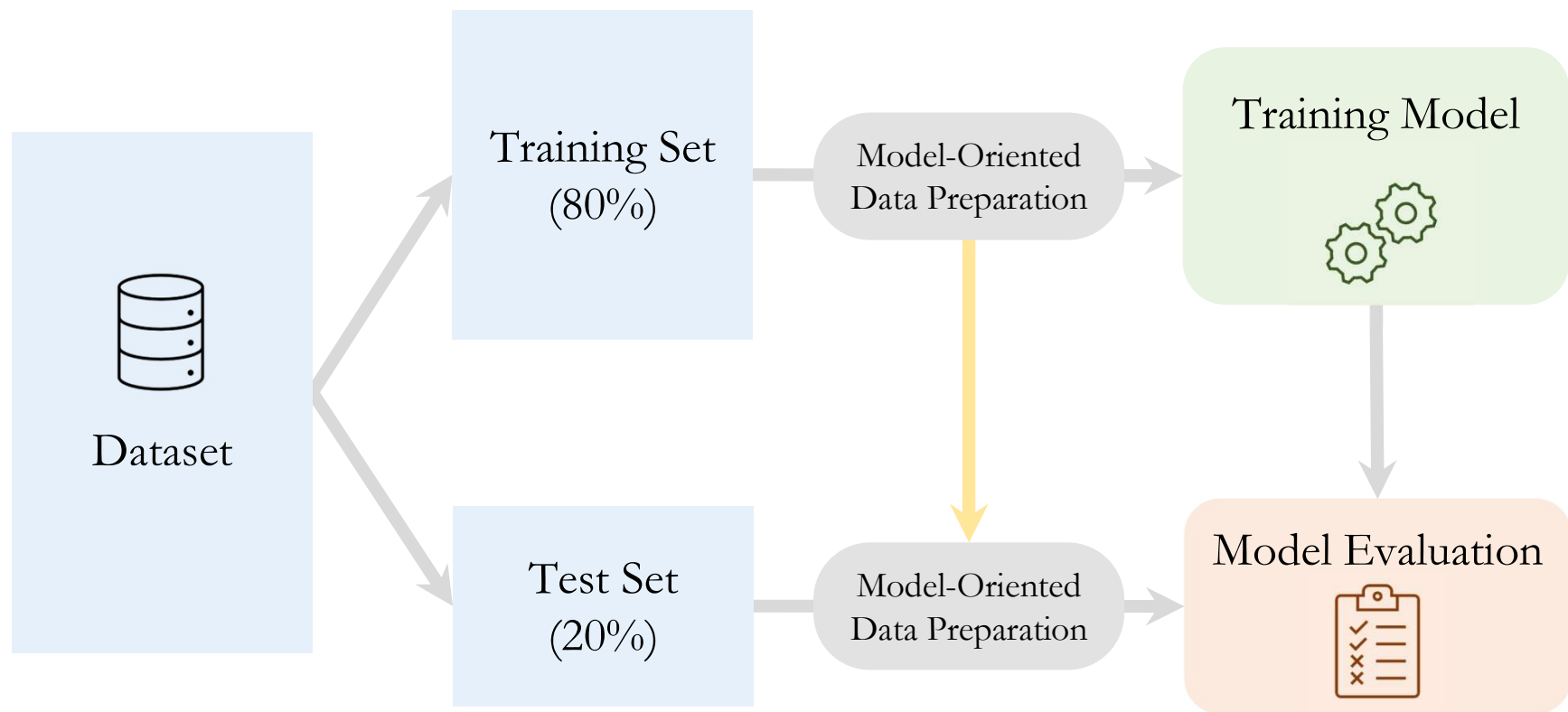
Bias measures how far off, on average, a model's predictions are from the true values.

Variance measures how much a model's predictions change with different training datasets.

Model Complexity: Overfitting vs Underfitting



Data Preparation and Modeling



k-Nearest Neighbors Algorithm

- The *k*-Nearest Neighbors (KNN) algorithm classifies a new record by looking at the most similar records in the training set.
- During training, KNN simply stores the available examples.
- When a new record needs to be classified, the algorithm finds the *k* closest examples and uses their labels to make a prediction. The new record is assigned to the class that is most common among its nearest neighbors.

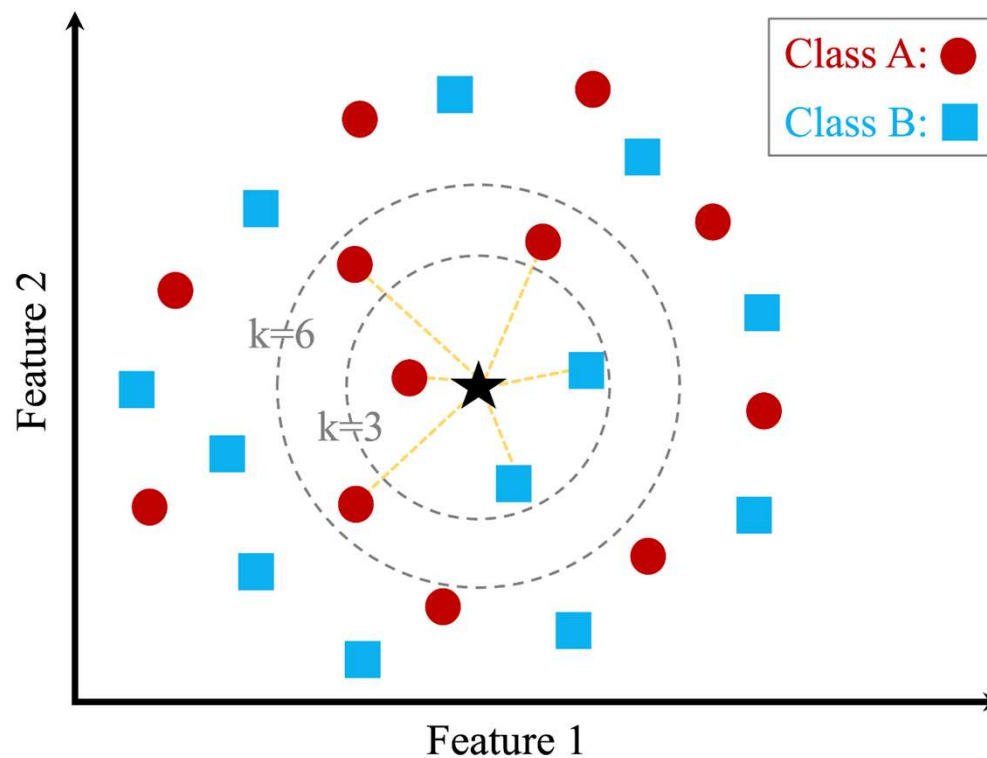
k-Nearest Neighbors Algorithm: Example

Suppose we are interested in classifying the type of drug a patient should prescribed. The training set consists of **200 patients** with **sodium/potassium (Na/K) ratio**, *age* and *drug prescribed*.

Our aim is to classify the type of drug a new patient should be prescribed that is:

- **Patient 1:** 40 years old with a Na/K ratio of 30.5.
- **Patient 2:** 28 years old with a Na/K ratio of 9.6.
- **Patient 3:** 61 years old with a Na/K ratio of 10.5.

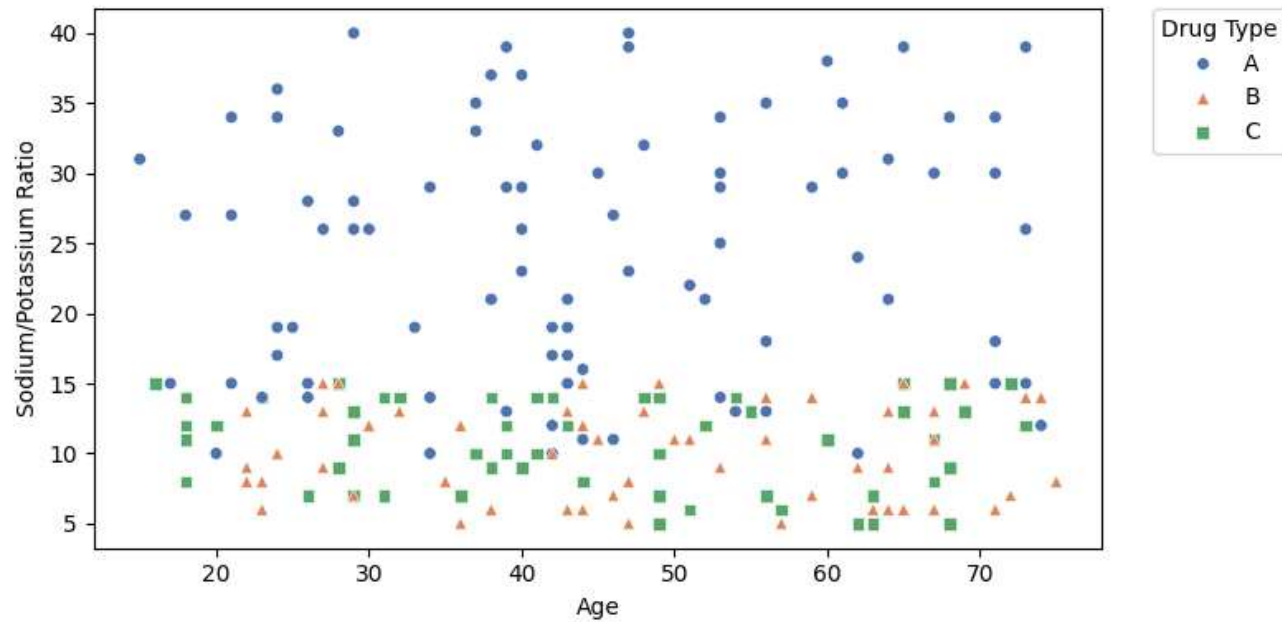
k -Nearest Neighbors Algorithm



For $k = 3$:
Selected class is **B**
with $2/3=66\%$
confidence

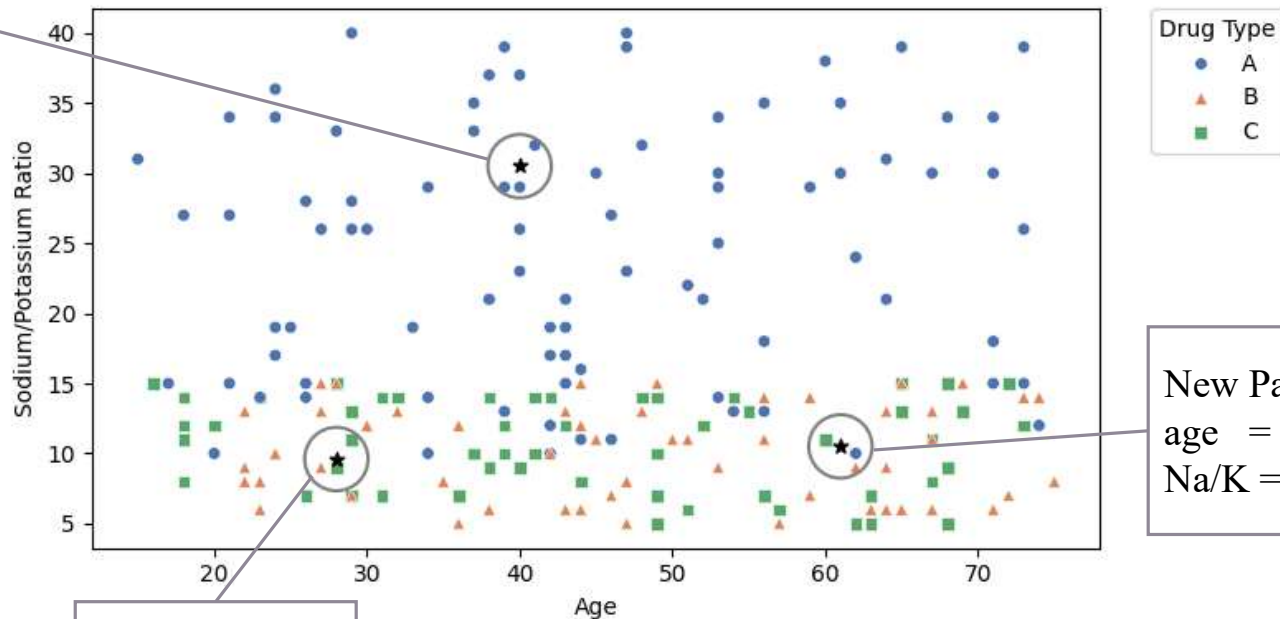
For $k = 6$:
Selected class is **A**
with $4/6=66\%$
confidence

k -Nearest Neighbors Algorithm



k-Nearest Neighbors Algorithm

New Patient 1
age = 40
Na/K = 30.5

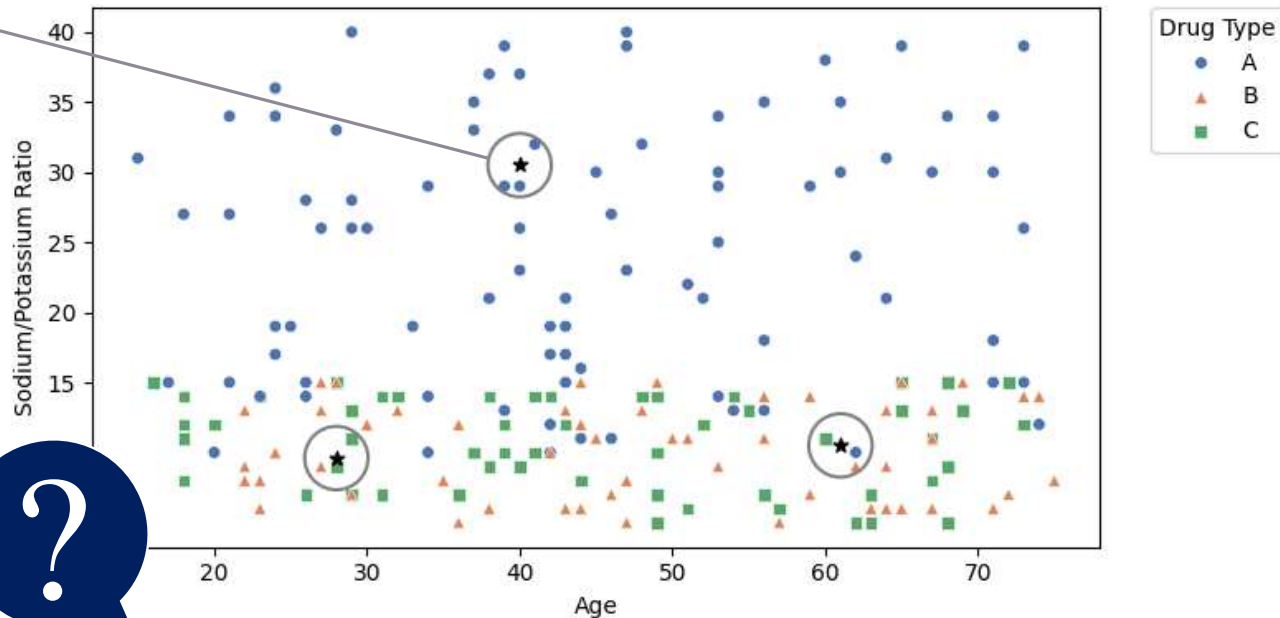


New Patient 2
age = 28
Na/K = 9.6

New Patient 3
age = 61
Na/K = 10.5

k-Nearest Neighbors Algorithm

New Patient 1
age = 40
Na/K = 30.5



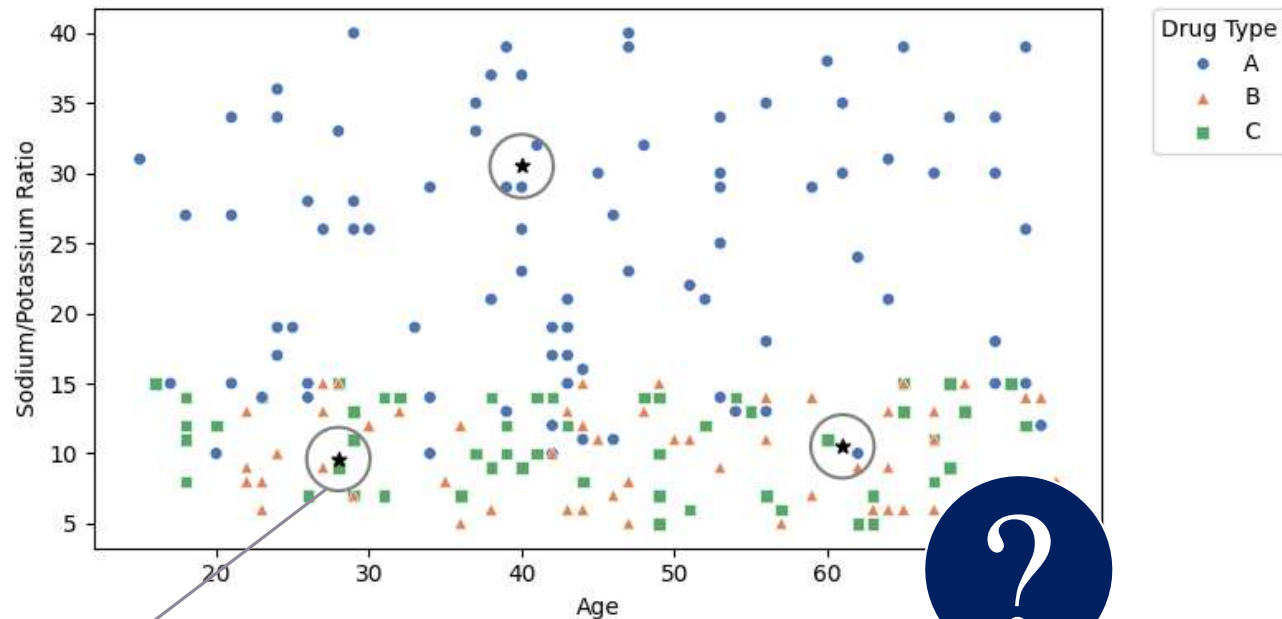
Which drug should **Patient 1** be prescribed?

k-Nearest Neighbors Algorithm

Patient 1

Since Patient 1's profile places them in the scatter plot near patients prescribed drug A, we classify Patient 1 as drug A. All points near Patient 1 are prescribed drug A, making this a straightforward classification.

k-Nearest Neighbors Algorithm



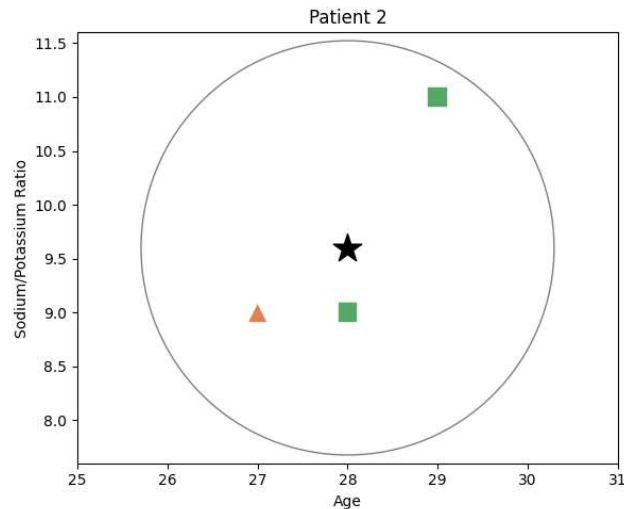
New Patient 2
age = 28
Na/K = 9.6

What can we say about **patient 2**?



k -Nearest Neighbors Algorithm

Close-up of three nearest neighbors to **New Patient 2**.



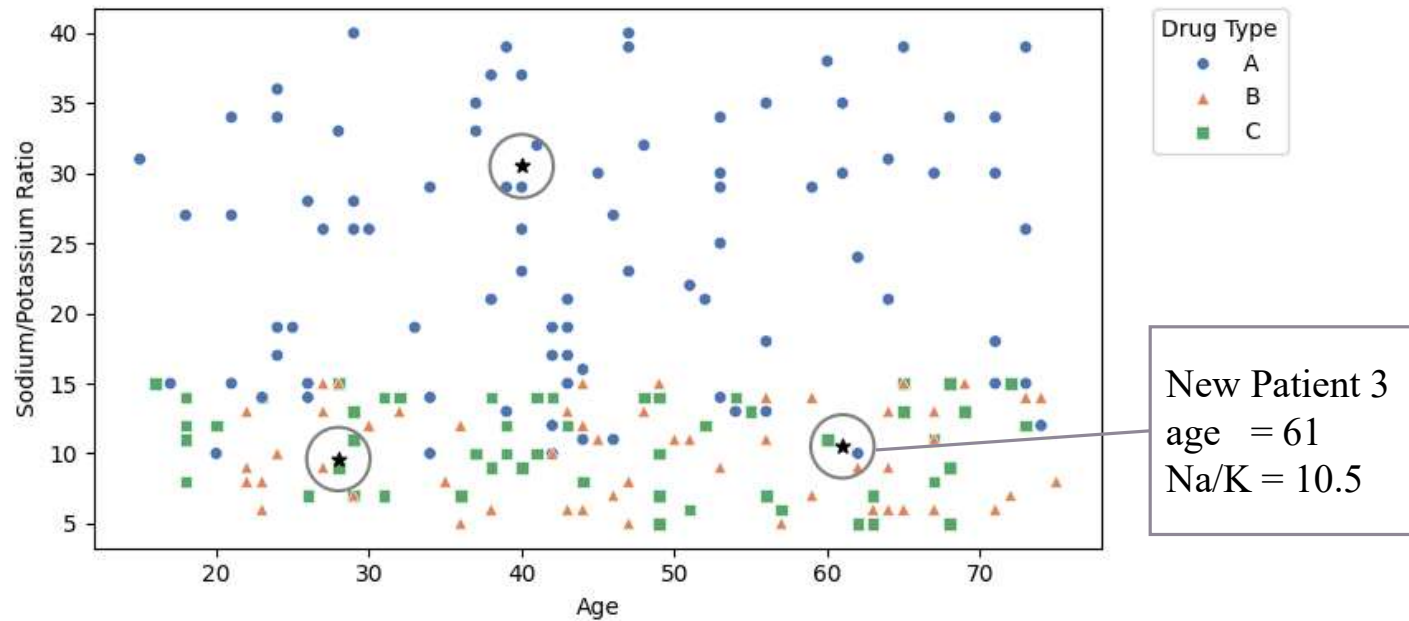
Patient 2 illustrates sensitivity to the choice of k

$k=1$: Highly sensitive to local variation.

$k=2$: The result is a tie.

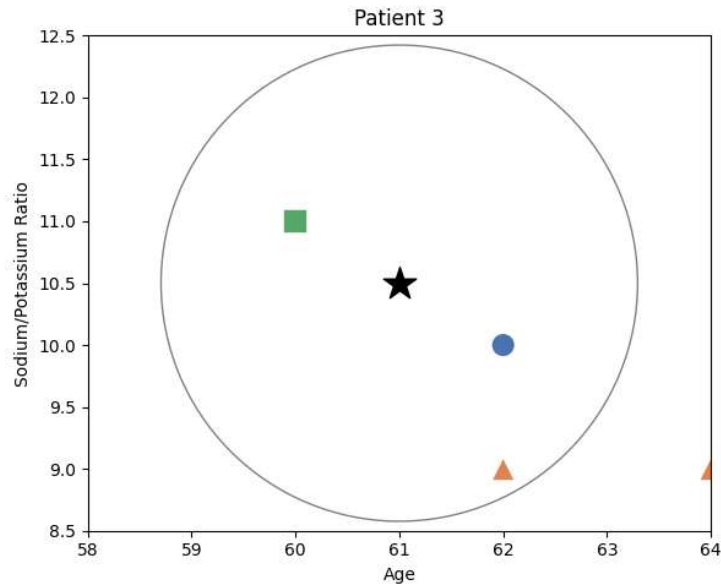
Note: The classification of Patient 2 differs based on the value chosen for k .

k -Nearest Neighbors Algorithm



k -Nearest Neighbors Algorithm

Close-up of three nearest neighbors to **New Patient 3**.

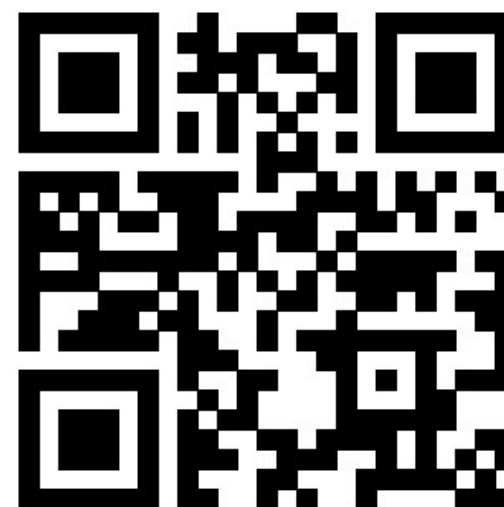


- With $k = 1$ or $k = 2$, Patient 3 is closest to both green and blue points. So, voting does not help.
- With $k = 3$, voting does not help since each of the nearest training points has a different classification.

Jupyter Notebook: creditcard.ipynb

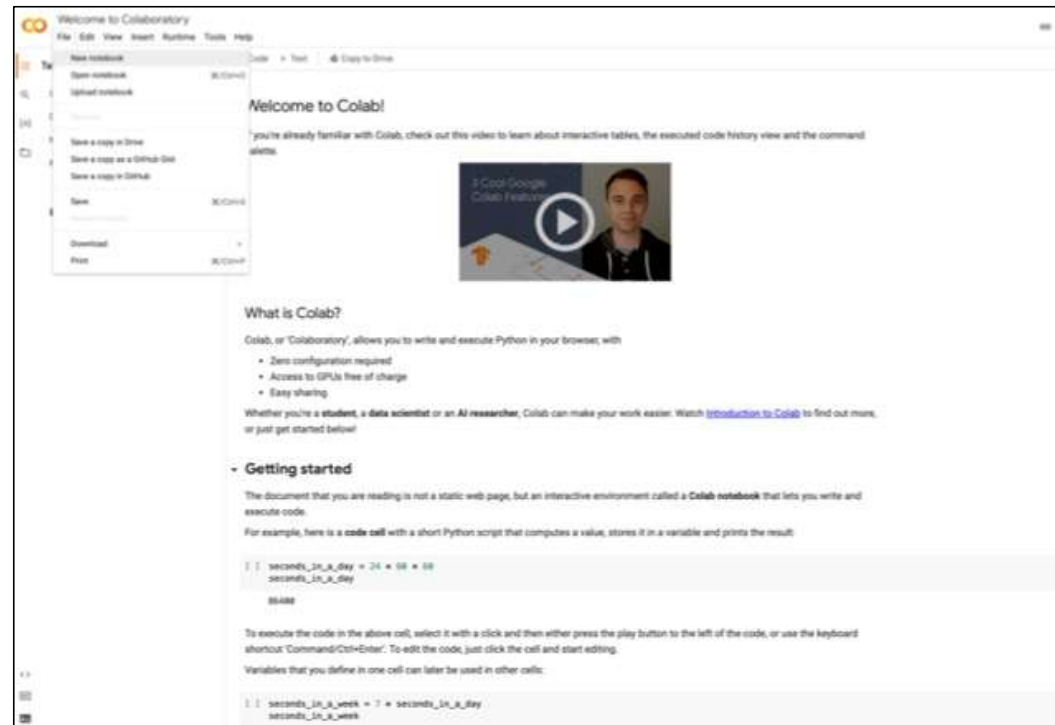
You are **not** expected to memorize this code.
Today we're learning how machine learning works and how to experiment with it, not Python syntax.

<https://github.com/vanraak/data2026>



edu.nl/y99yd

Google Colab



colab.research.google.com

Advantages and Disadvantages of k -Nearest Neighbors

Advantages:

- No training phase required
- Easy to implement and interpret
- Non-parametric: no assumptions about distribution of the data

Disadvantages:

- Hard to determine "correct" k
- Computationally inefficient with large and multi-dimensional datasets
- Sensitive to outliers
- Need to normalize data

Evaluation

Examples of evaluation methods:

- Confusion Matrix
- Metrics:
 - Accuracy
 - Precision
 - Recall
- ROC / AUC

Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions made by the classification model compared to the actual outcomes (target value) in the data. The matrix is $N \times N$, where N is the number of target values (classes). The following table displays a 2×2 confusion matrix for two classes (Negative and Positive):

		Predicted Values	
		Negative	Positive
Actual Values	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

Like the hypothesis testing, you may recall the two errors we defined as type-I and type-II.

Metrics

- **Accuracy:** Measures the proportion of all predictions that are correct.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

- **Precision:** Measures the proportion of predicted positives that are actually positive.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Important when the cost of false positives is high (e.g., important mail classified as spam or false medical diagnoses leading to unnecessary treatments and emotional distress)

- **Recall:** Measures the proportion of actual positives that are correctly identified.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Important when cost of false negatives is high (e.g., missing fraud)

		Predicted Values	
		Negative	Positive
Actual Values	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

		Predicted Values	
		Negative	Positive
Actual Values	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

		Predicted Values	
		Negative	Positive
Actual Values	Negative	True Negative (TN)	False Positive (FP)
	Positive	False Negative (FN)	True Positive (TP)

ROC curve

An ROC curve (receiver operating characteristic curve) is a graph showing the performance of a binary classification model at all classification thresholds. This curve plots two parameters:

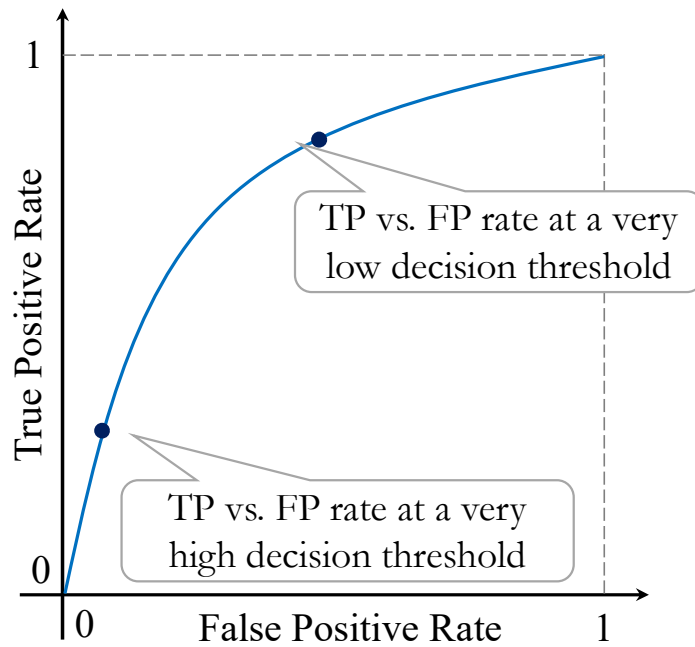
FPR: Out of all *actual negatives*, how many did we incorrectly label as positive?

$$\text{False Positive Rate} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

TPR: Out of all *actual positives*, how many did we correctly catch?

$$\text{True Positive Rate} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

ROC curve

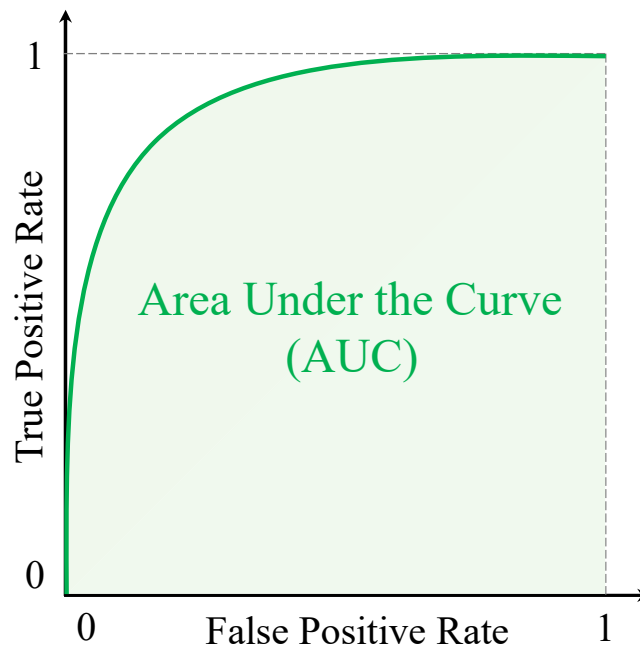


$$\text{False Positive Rate} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

$$\text{True Positive Rate} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

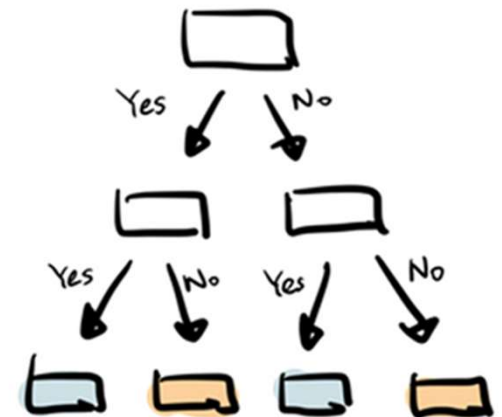
Area Under the Curve (AUC)

The area under the ROC curve is often a measure of the quality of the classification models. A random classifier has an area under the curve of 0.5, while AUC for a perfect classifier is equal to 1. In practice, most of the classification models have an AUC between 0.5 and 1.



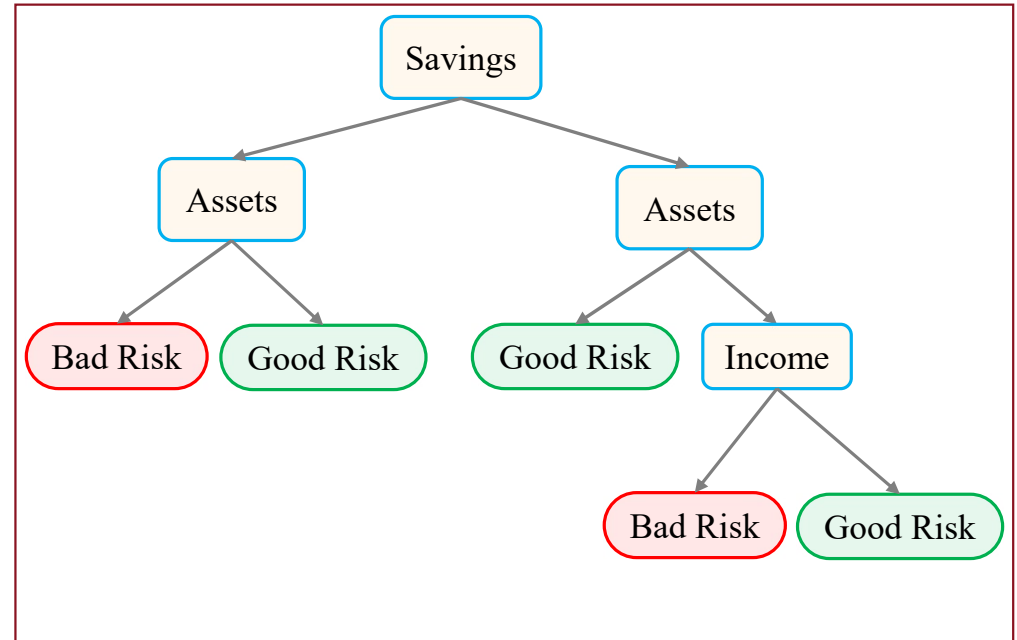
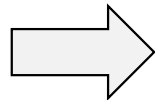
Decision Trees

- Employ a divide-and-conquer method.
- Decision tree builds tree branches in a hierarchy approach.
- Each branch can be considered as an if-else statement.
- The branches develop by partitioning the dataset into subsets based on most important features.
- Final classification happens at the leaves of the decision tree.

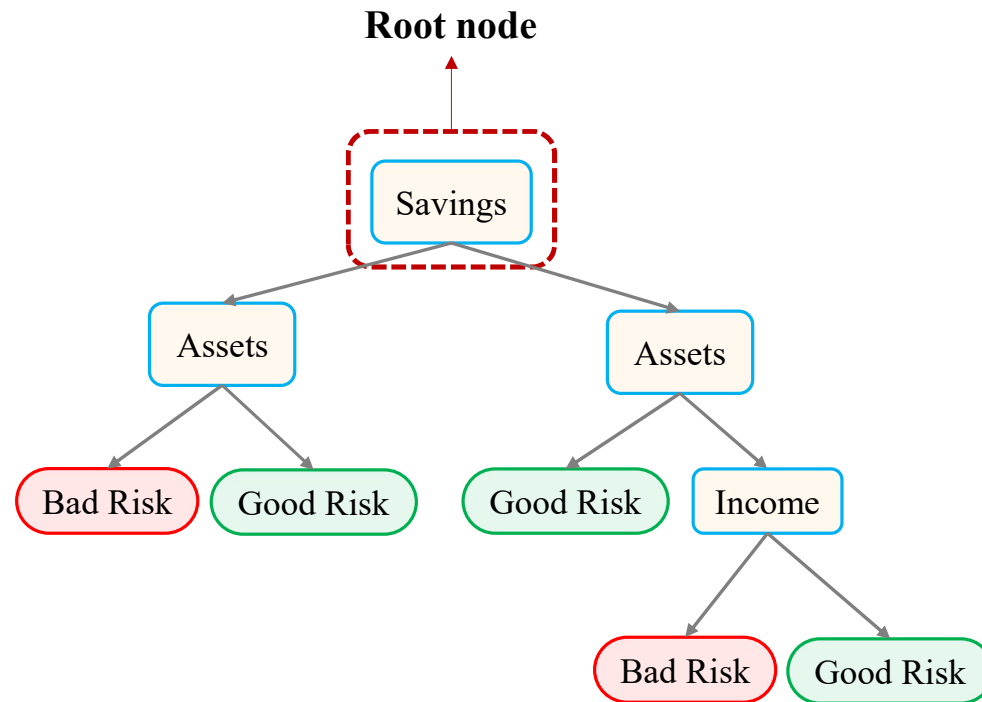


Decision Trees

	Features			Target
	Savings	Assets	Income	Label
0	Low	Low	Low	Bad Risk
1	Low	Low	High	Bad Risk
2	Low	High	Low	Good Risk
3	Low	High	High	Bad Risk
4	Low	High	High	Good Risk
5	Low	Low	Low	Bad Risk
6	High	Low	Low	Good Risk
7	High	Low	High	Good Risk
8	High	High	Low	Bad Risk
9	High	High	High	Good Risk
10	High	High	Low	Good Risk
11	Low	Low	High	Bad Risk
12	High	Low	Low	Good Risk
13	High	High	High	Good Risk
14	Low	High	Low	Good Risk
15	Low	Low	High	Bad Risk
16	High	Low	High	Good Risk
17	Low	High	High	Good Risk

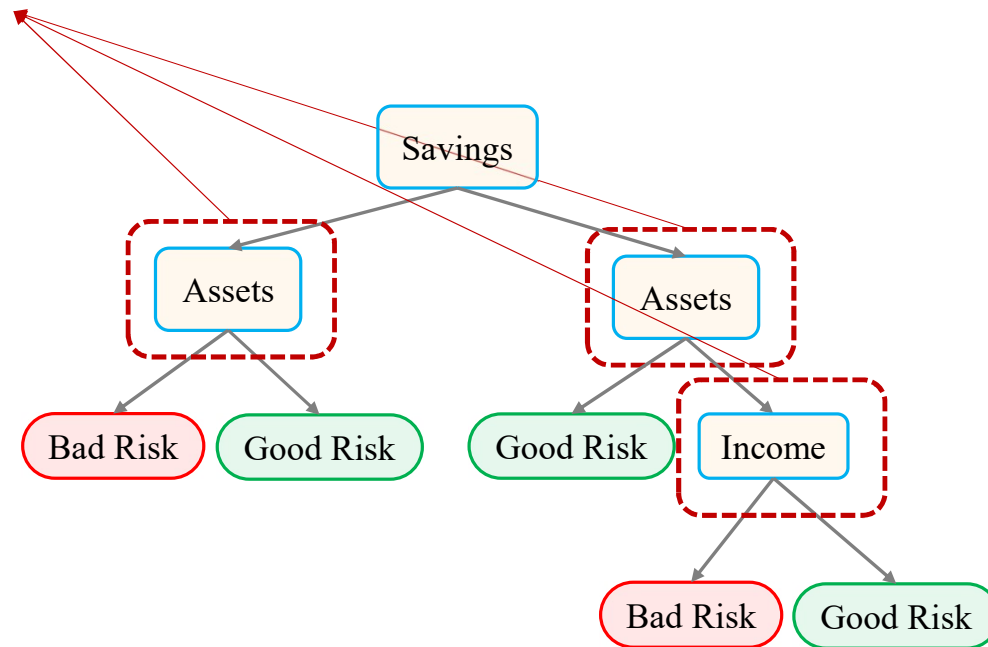


Decision Trees



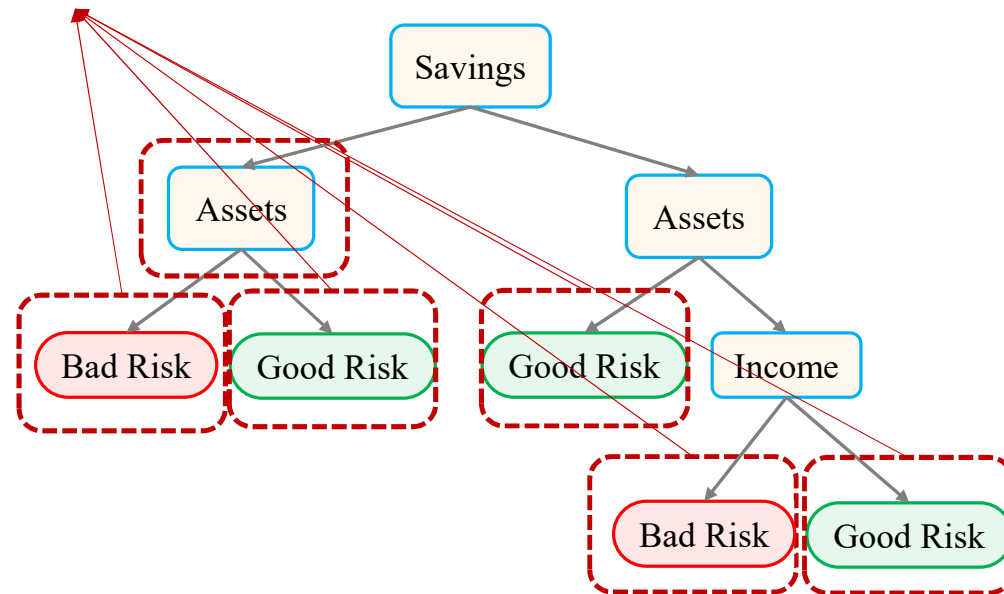
Decision Trees

Decision nodes



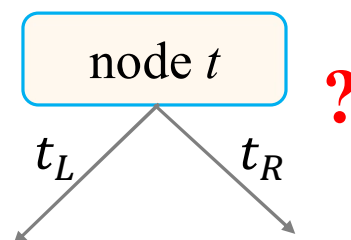
Decision Trees

Leaf nodes



Decision Tree Algorithm

- Splits at decision nodes are strictly **binary**, resulting in two branches.
- The algorithm grows the tree by evaluating all predictor variables and choosing the optimal split according to the **largest reduction in impurity**.
- Repeats this process recursively on each resulting subset until no further split improves purity or a stopping condition is reached.



Splitting Criteria

- Gini impurity
 - Measures probability of misclassification
 - Interprets how often a random label would be wrong
 - Used as default in scikit-learn
- Entropy: Measure of uncertainty
 - Measures uncertainty in the data
 - From information theory
 - Based on information gain
- Key idea:
 - Both measure node impurity (how mixed classes are)
 - Both aim to create more pure child nodes after splits
 - Usually produce very similar trees and results
 - Gini is slightly faster to compute
 - Entropy has a stronger theoretical foundation

Decision Tree

Advantages:

- Intuitive and easy to explain
- Do not require normalization of data
- Robust to missing data and outliers
- Non-parametric (no assumptions about underlying distributions)

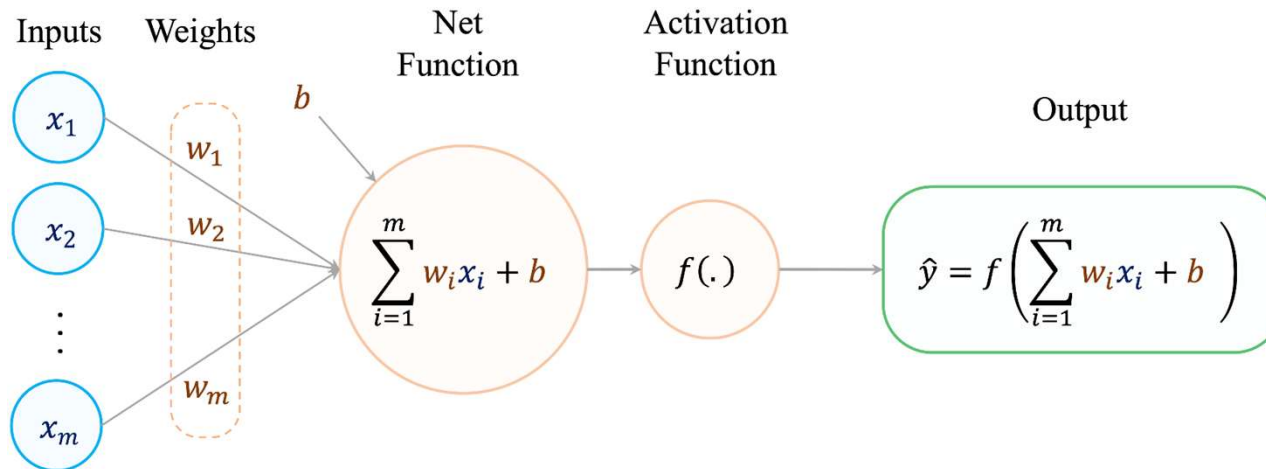
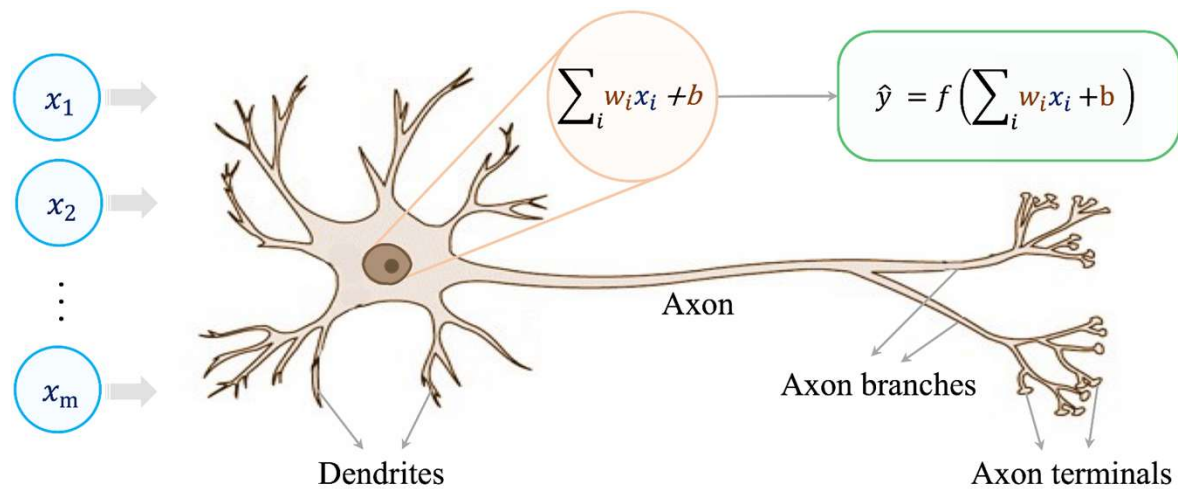
Disadvantages:

- Need to control tree complexity (prone to overfitting)
- New or changed data can affect the structure of the decision tree considerably
- Can require significant resources to train
- Greedy algorithm: can result in non-optimal solutions

Neural Networks

- Inspired by complex learning systems recognized in brain
 - *Single neuron has simple structure*
 - *Interconnected sets of neurons perform complex learning tasks*
- Artificial Neural Networks attempt to replicate non-linear learning found in nature
- The ability of neural networks to discover patterns allows them to far surpass traditional methods in computer science, making them viable for use in research and business

Neural Networks



Activation Function

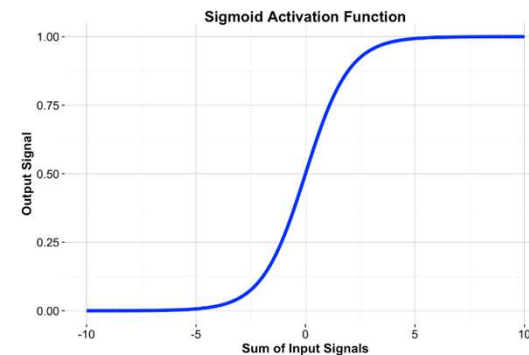
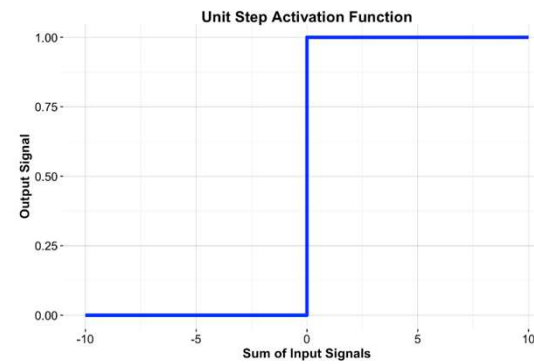
One option for f could be a simple Threshold function:

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Sigmoid Function: combines nearly linear, curvilinear, and nearly constant behavior depending on input value:

$$f(x) = \frac{1}{1 + e^{-x}}$$

This function returns values $[0, 1]$.



Activation Functions in Neural Networks

While the sigmoid function was foundational in early neural networks, modern architectures benefit from activation functions that offer faster convergence and improved gradient flow. The most widely used alternatives are:

- *Hyperbolic Tangent (tanh)*: Like sigmoid, but outputs values between -1 and 1, making it zero-centered and often better suited for hidden layers.
- *ReLU (Rectified Linear Unit)*: Defined as:

$$f(x) = \max(0, x)$$

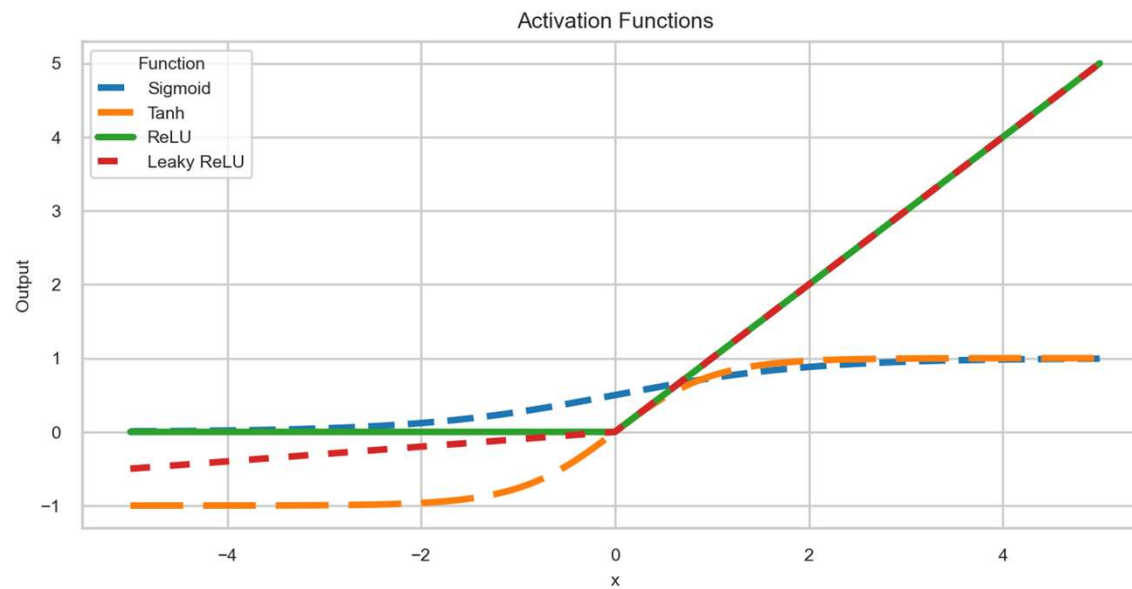
ReLU is computationally efficient and helps reduce the vanishing gradient problem.

- *Leaky ReLU*: A variation of ReLU that allows a small, non-zero gradient for negative input values, helping prevent the "dying ReLU" problem. It is defined as:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

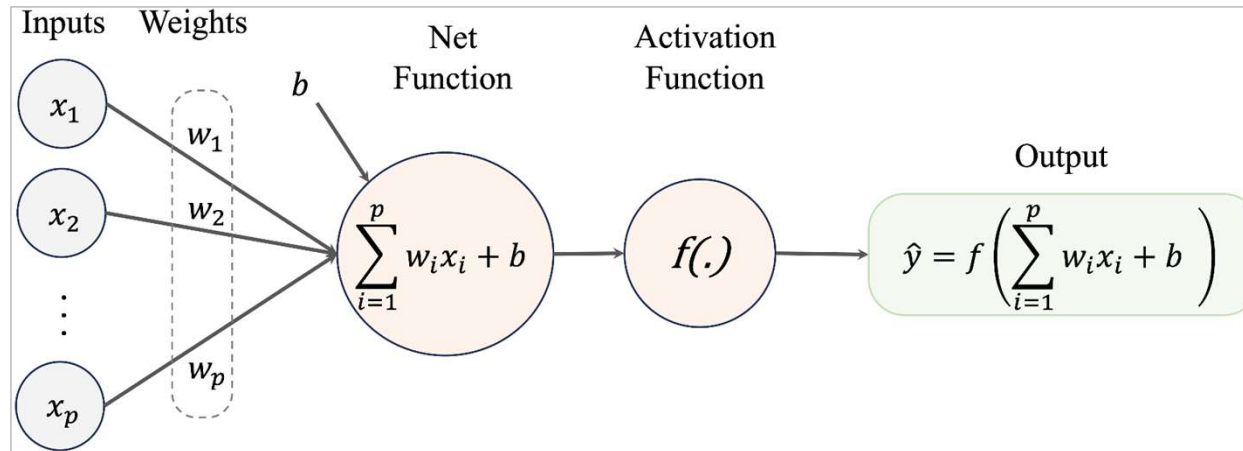
where α is a small constant (commonly 0.01). Unlike ReLU, Leaky ReLU ensures neurons continue to learn even when their inputs are negative.

Activation Functions in Neural Networks



How does a Neural Network learn?

Each node in the network is a neuron having their weights. These weights must be “*learned*” by the model to give the results closest to the training data. This happens through processes called **Back-propagation** and **Gradient Descent**.



Once, we have “trained” the model, ie learned the optimum weights for all neurons, the input data is sent into the Neural Network, each neuron multiplying its inputs and weights, applying the activation on the sum and sending it to the next layer until we have the final result.

Advantages and Disadvantages of Neural Networks

Advantages:

- Suitable for highly complex settings
- Can handle unstructured data
- Parallel processing
- Can be used for generative purposes

Disadvantages:

- "Black box" nature
- Proneness to overfitting
- Expensive and time-consuming to train
- Sensitive to noise

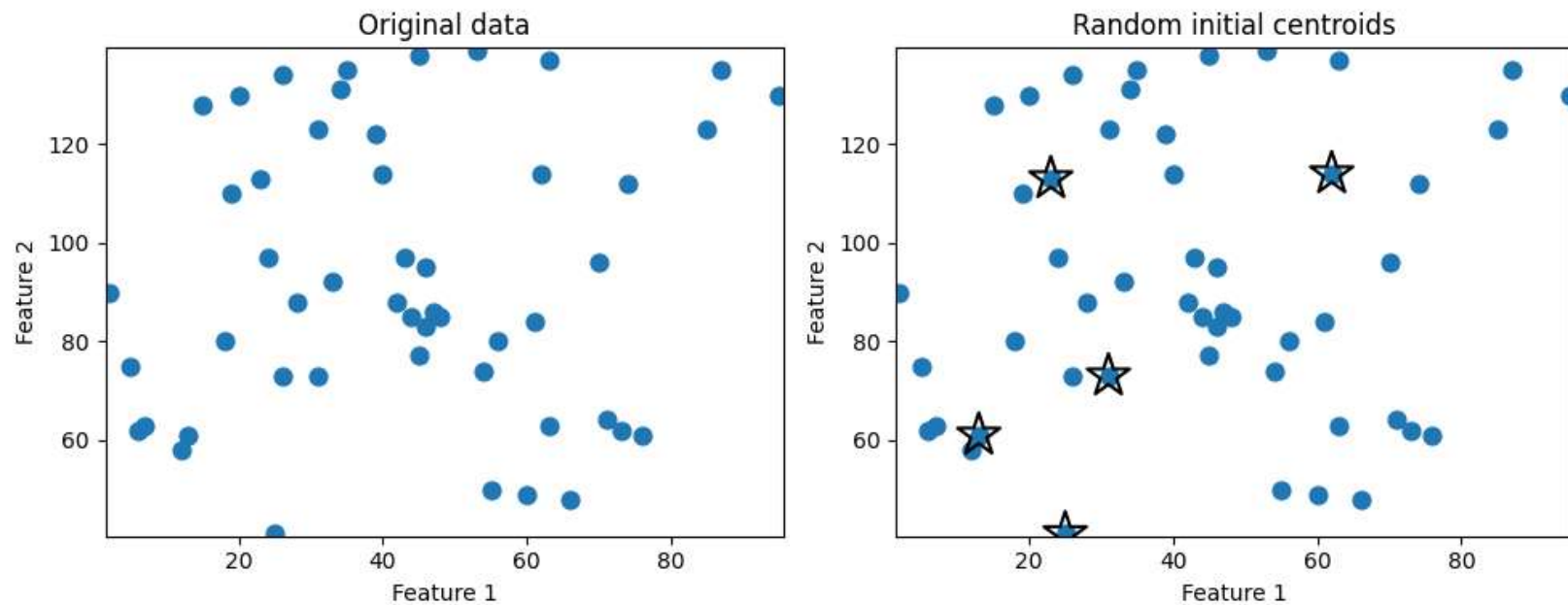
k-Means Clustering

- Unsupervised machine learning technique
- Objective: create k clusters
- No labels required
- Greedy algorithm

Steps:

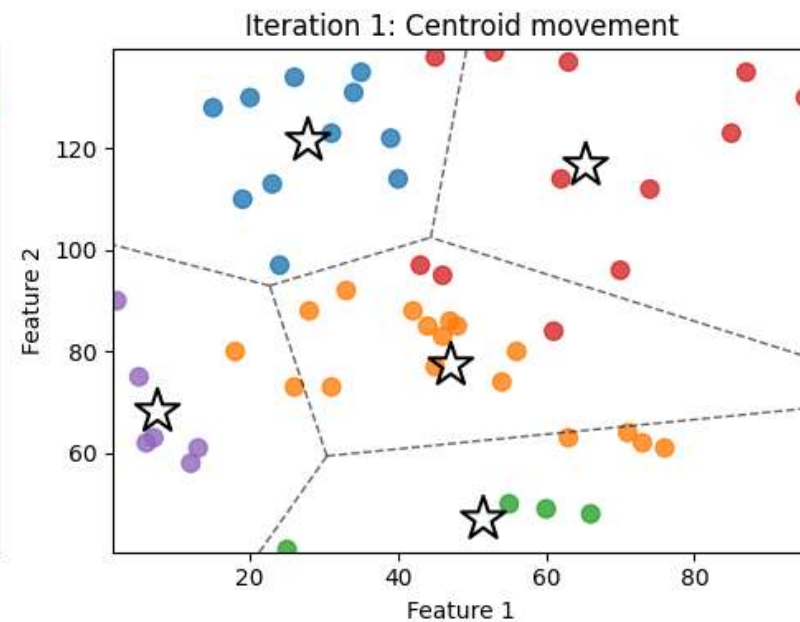
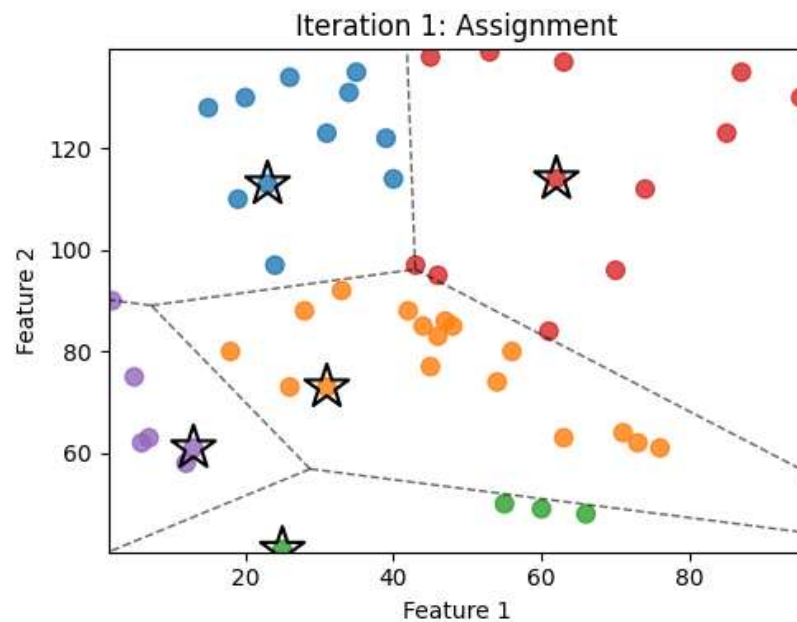
1. Determine number of clusters (k) to be generated
2. Select k random starting points as initial centroids
3. Select observations closest to each centroid and assign them to a cluster
4. Compute new centroids (center of each cluster)
5. Repeat steps 3 and 4 until centroids stop moving

Initial Centroids



Five observations are randomly chosen as initial centroids.

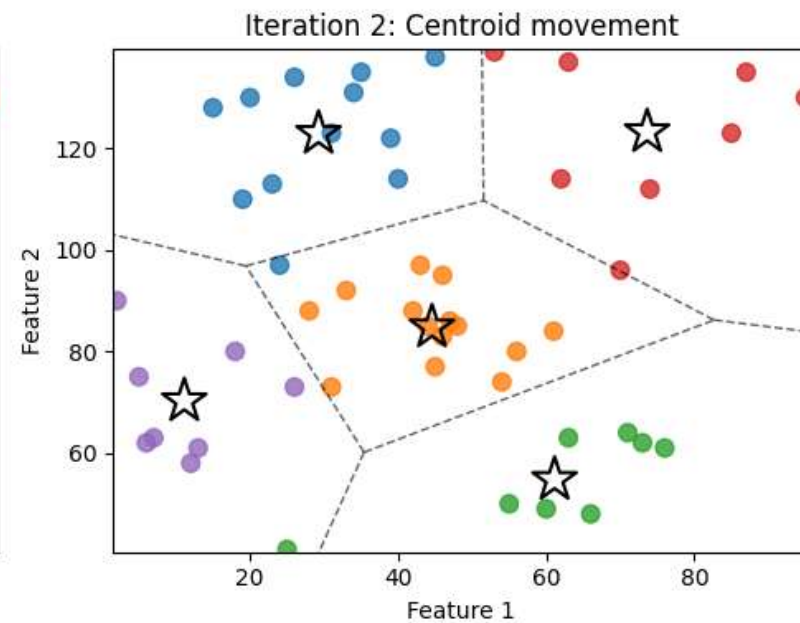
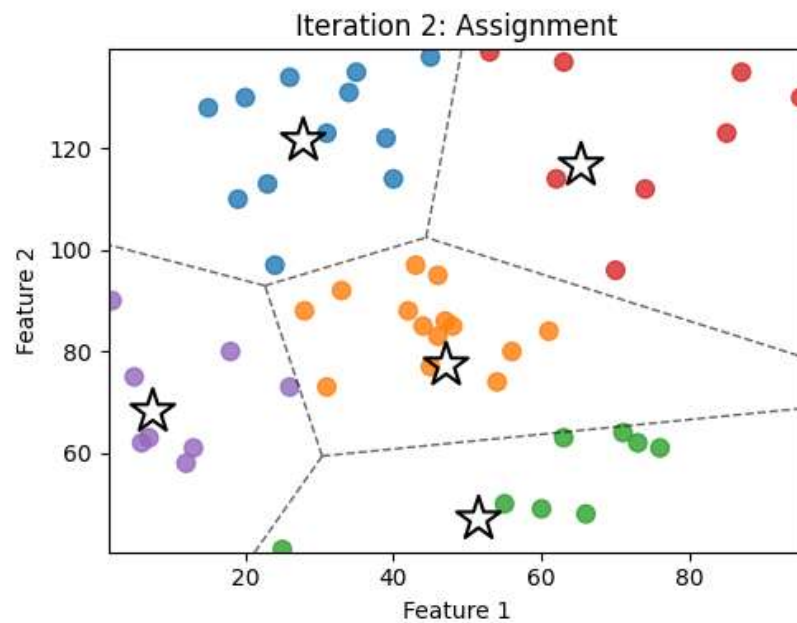
Iteration 1: Assignment and Centroid Movement



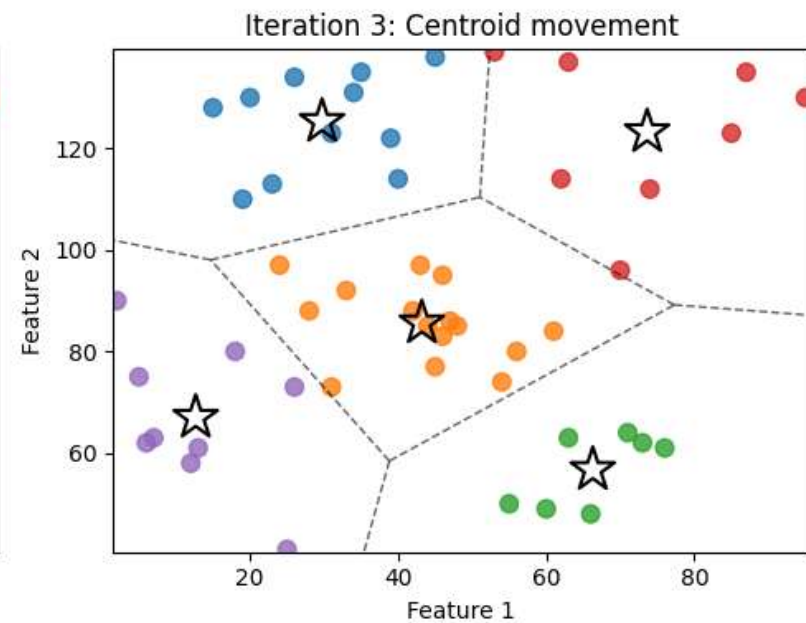
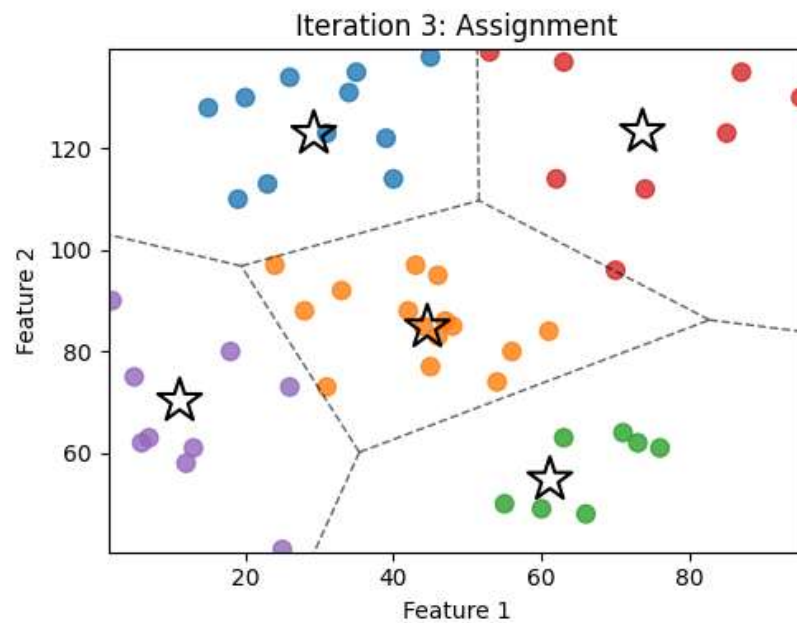
1: Each observation is assigned to its nearest centroid, partitioning the feature space into Voronoi regions defined by proximity to the current centroids.

2: Centroids are updated by moving them to the mean position of the observations assigned to each cluster.

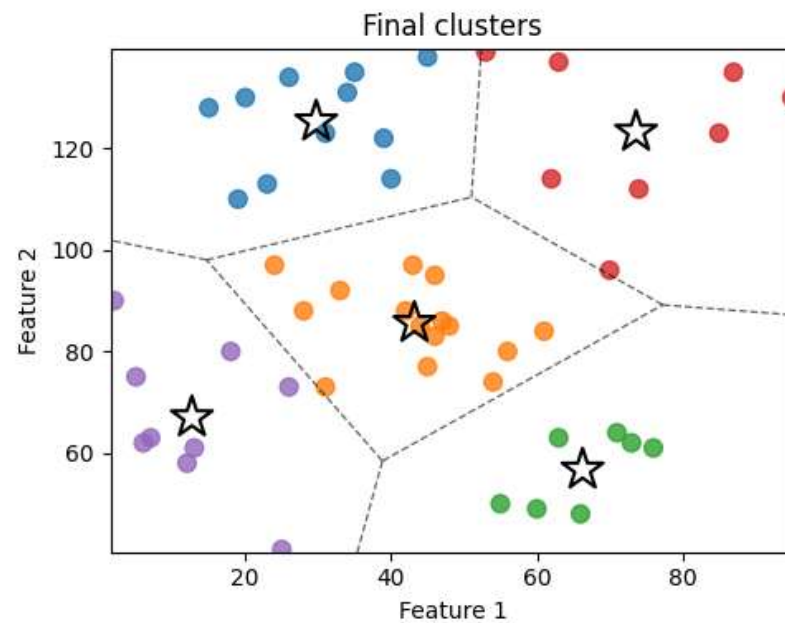
Iteration 2: Assignment and Centroid Movement



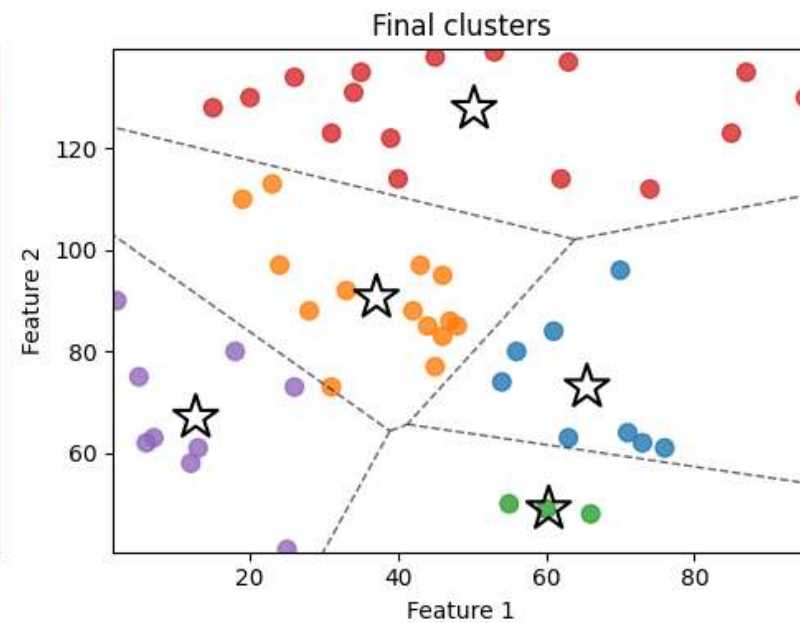
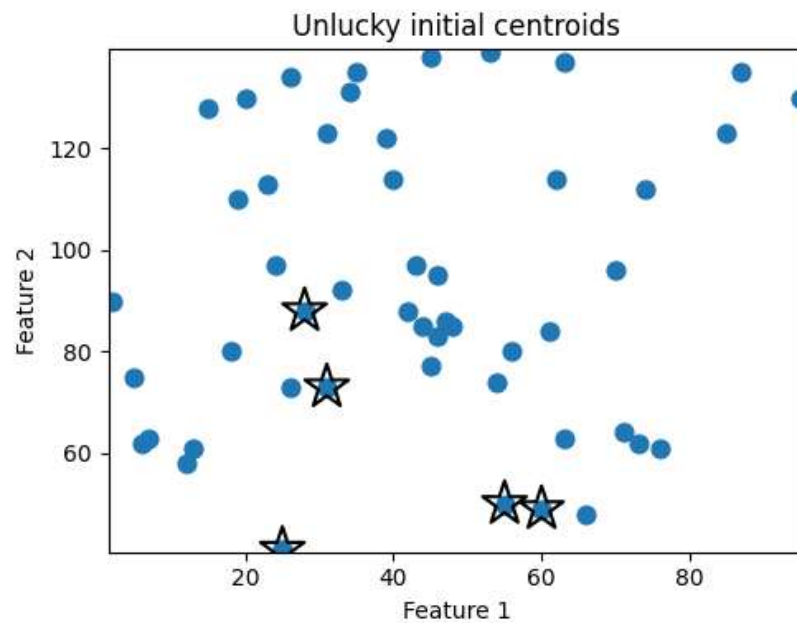
Iteration 3: Assignment and Centroid Movement



k -Means Clustering: Final Clusters



k-Means Clustering: Impact of Unlucky Centroids



Advantages and Disadvantages of k -Means

Advantages:

- Easy to understand and implement
- Fast algorithm: scales well with large datasets

Disadvantages:

- Depending on the setting, determining the right number of clusters may be subjective
- Greedy algorithm: results dependent on initial centroids
- Size based measures are sensitive to relative scales of features: may need to normalize data (scale: 0-1)

References

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. *SPSS inc*, 9(13), 1-73.